

DevRel-DeepAgent

*Build an Elasticsearch subagent with
Langchain's Deep Agents AI harness*



metrics



sentiment



web



elastic

The roadmap

Where the time goes and where you'll be coding



Half the time is hands-on.

Slides set context; the workshop happens in your editor.

Setup check

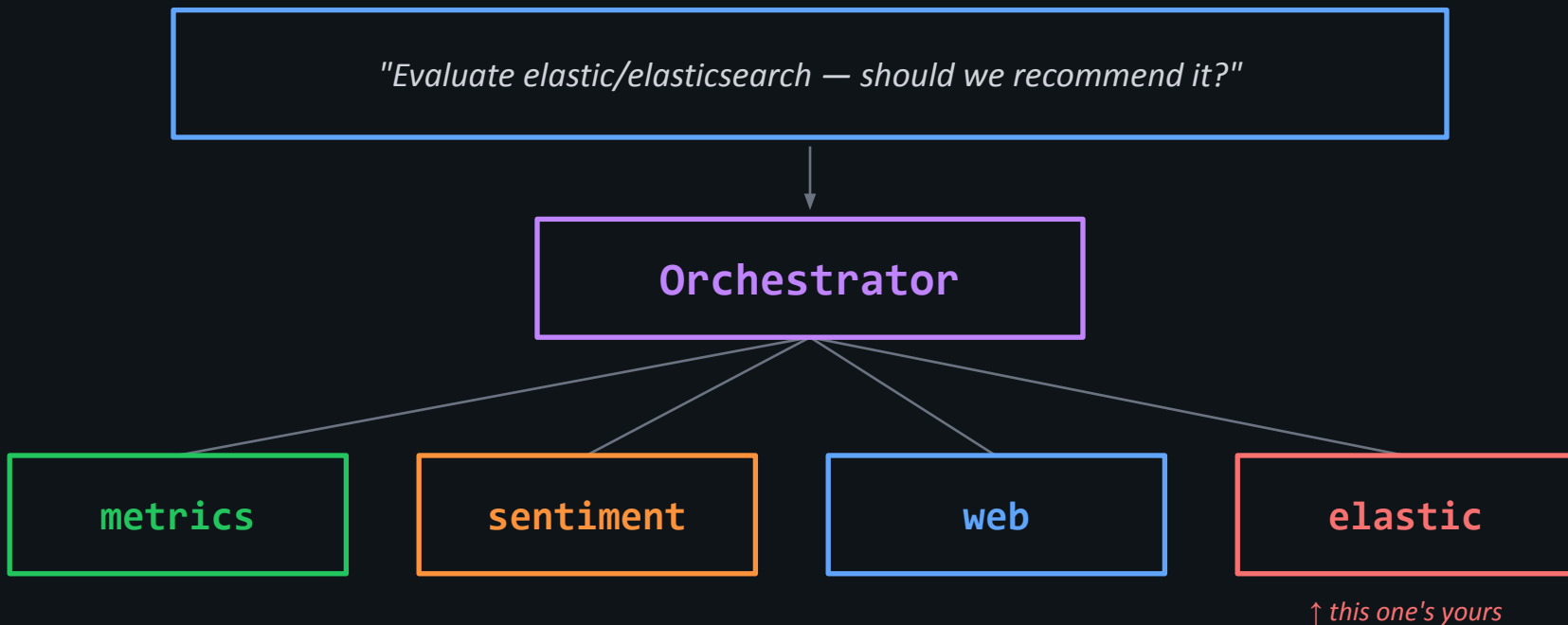
Right now, in your terminal:

```
$ cd ~/deepagent_workshop_py  
$ python scripts/verify.py
```

```
✓ env: LITELLM_API_KEY          set  
✓ env: LITELLM_API_BASE        set  
✓ ... 12 more env + service checks set  
  
✓ All 14 checks passed. You're ready.
```

If anything is red – raise your hand now.

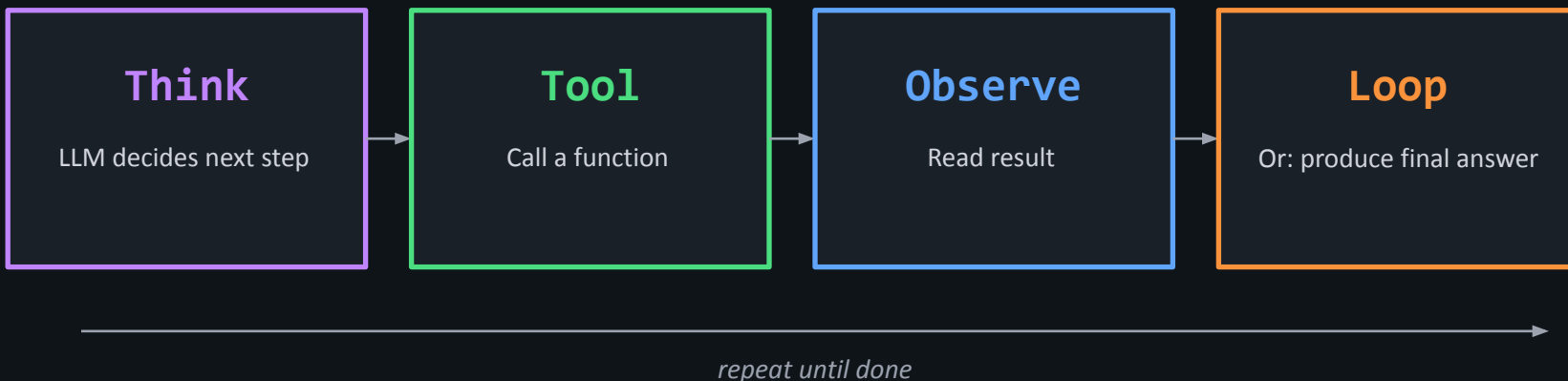
What we're building



Four specialists, one orchestrator, one final report.

The agent loop

Every agent, deep or shallow, is just this



Tools are just functions the LLM can call. The loop is just a while-loop you wrote.

Why subagents?

Three concrete reasons

Specialization

Each subagent has its own prompt, model, and tool subset. Better at its job than a generalist with 20 tools.

Context isolation

Subagent state stays in its loop. The orchestrator only sees the final reply — its context window stays clean.

Parallelism

Independent subagents fan out concurrently. Four subagents, ~3× faster than serial.

A subagent is just a dict

Four fields. That's it.

```
elastic_subagent = {  
    "name": "elastic-agent",  
  
    "description": (  
        "Use to retrieve cached/historical research data "  
        "from Elasticsearch via semantic search."  
    ),  
  
    "prompt": elastic_prompt,    # loaded from prompts/elastic.md  
  
    "tools": [ask_elastic_agent],  
}
```

name

unique identifier the
orchestrator routes on

description

WHEN to delegate — read by
orchestrator's LLM

prompt

this subagent's brief — its
role, tools, format

tools

what this subagent
can call

What is Elasticsearch?

Distributed JSON search + analytics engine

Full-text search

Lightning-fast full-text search over structured/unstructured JSON documents.

Multi-modal search

Lexical (BM25), structured filters, geospatial, vector — all in one query.

Massive scale

Sharded across nodes or serverless. Used by Netflix, Uber, GitHub for log/metric/search workloads.

Today: we use it for semantic search over a research corpus.

semantic_text + ELSER

Auto-embeddings on save, auto-vector-search on query

```
// index mapping
{
  "mappings": {
    "properties": {
      "semantic_content": {
        "type": "semantic_text"
      }
    }
  }
}
```

On insert:

Elastic auto-embeds the field with ELSER (Elastic's sparse encoder). No code from you.

On query:

`semantic_content : ?term`
→ vector search ranked by relevance.

First write triggers ELSER auto-deploy (~30-60s). After that, it's instant.

ES|QL – pipe-syntax queries

SQL-ish, but designed for streaming pipelines

```
FROM technology-research METADATA _score
| WHERE semantic_content : ?description
| SORT _score DESC
| KEEP repo, timestamp, tags, analysis.viability_score, analysis.summary, _score
| LIMIT ?limit
```

This is the actual ES|QL query our Kibana Agent Builder tool uses.

FROM

the index

WHERE

field : value =
semantic match

SORT

by relevance score

KEEP

only return
these columns

LIMIT

top N results

Kibana Agent Builder

Agents that wrap ES|QL tools, exposed as a REST endpoint

POST /api/agent_builder/converse

```
{"input": "find similar..."}
```

Agent Builder LLM

natural language → ES|QL choice

ES|QL tool

find-similar-technologies

Elasticsearch

ELSER ranks; returns top N

What we configured for you

technology-research

Index seeded with 10 curated technology entries (LangGraph, AutoGen, Elasticsearch, Loki, OpenTelemetry, ...). `semantic_text` on `semantic_content`.

find-similar-technologies

ES|QL tool registered with Kibana Agent Builder. Takes (description, limit), returns top-N matching repos.

technology-research-agent

Kibana agent attached to the tool above + a connector pointing at the LiteLLM proxy. Has an agent ID — that's your `ELASTIC_AGENT_ID`.

Let's play a number game!

<https://tinyurl.com/SaveGorst>

What you'll build, file by file

Two stubs, reverse order: subagent shell first, tool body second

Build A

`src/subagents/elastic.py`

Subagent definition

Plain dict with `{name, description, prompt, tools}`

Build A

`src/prompts/elastic.md`

System prompt

Markdown doc the subagent's LLM reads as its instructions

Build B

`src/tools/ask_elastic_agent.py`

LangChain tool wrapper

Wraps the `kibana_client` as a `@tool` with a docstring

Pre-provided: `src/tools/kibana_client.py` (HTTP client) — already done for you.

Build A • Elastic subagent + prompt

Two files. Already imported by the orchestrator.

Open: `src/subagents/elastic.py`

TODO 1 — load `src/prompts/elastic.md` from disk

TODO 2 — define `elastic_subagent` dict (name, description, prompt, tools)

Open: `src/prompts/elastic.md`

Write the system prompt: identity, mission, tool guidance, output format, honesty rule. Use the next slides as reference.

```
# When you're done, restart the dev server and ask a query:  
$ python -m server.app  
# Open localhost:3000, click "Run agent".  
# The elastic-agent panel lights up red — and finishes.
```

Build A • TODOs 1 & 2

```
elastic_prompt = (Path(__file__).parent.parent / "prompts" / "elastic.md").read_text()

elastic_subagent: dict = {
    "name": "elastic-agent",
    "description": (
        "Use to retrieve cached or historical research data from "
        "Elasticsearch: prior research reports, snapshots, trends, "
        "adoption signals, and similar technologies via semantic "
        "search. Always check elastic-agent BEFORE triggering "
        "fresh data collection from other subagents."
    ),
    "prompt": elastic_prompt,
    "tools": [ask_elastic_agent],
}
```


Build A • prompt structure

Five sections. Use the same voice the other subagents use.

1 • Identity

"You are an Elastic Data Specialist." One sentence.

2 • Mission

What this subagent is responsible for. Mention it asks the Elastic Agent — doesn't write ES|QL itself.

3 • Tool guidance

How to use ask_elastic_agent: full repo names, time ranges, 3-4 example queries.

4 • Output format

Markdown skeleton: ## Elasticsearch Research Summary, ### Data Retrieved, ### Key Findings, ### Gaps.

5 • Honesty rule

Never fabricate results. If the agent returns no data, say so clearly.

Build B • ask_elastic_agent tool

src/tools/ask_elastic_agent.py — wrap the kibana_client as a @tool

Open: `src/tools/ask_elastic_agent.py`

TODO 1 — function signature: query (required), conversation_id (optional)

TODO 2 — handler: read ELASTIC_AGENT_ID, call converse(), return text

TODO 3 — docstring: what this tool does, when to use it

```
# When you're done, run:  
$ python scripts/test_tool.py
```

✓ Build B health check passed.

Build B • the function body

```
@tool
def ask_elastic_agent(query: str, conversation_id: str = "") -> str:
    """Send a natural-language request to the Elastic Agent and
    return its response. The agent has access to ES|QL tools for
    semantic search, historical snapshots, trend data, ..."""

    agent_id = os.environ.get("ELASTIC_AGENT_ID")
    if not agent_id:
        raise RuntimeError("Missing ELASTIC_AGENT_ID in .env")

    resp = converse(ConverseRequest(
        agent_id=agent_id,
        input=query,
        conversation_id=conversation_id or None,
    ))
    return resp.text
```

What you built

3 files • ~50 lines • 1 deep agent

Concepts you walked away with:

- the agent loop
- subagent orchestration via the task tool
- semantic_text + ELSER for vector search without an embedding pipeline
- ES|QL pipe queries with parameter substitution
- Kibana Agent Builder as a NL → ES|QL gateway

Where to take it next

Add a 5th subagent

e.g. a CVE/security scanner. Same pattern: tool + prompt + register.

Persist research

Have the orchestrator write each report into the technology-research index. Later runs benefit.

Stream tokens

Pipe `on_chat_model_stream` events to the UI for typewriter-style output.

Skip Agent Builder

Have the elastic subagent build ES|QL itself; remove the middle LLM call.

Resources

Workshop repo

github.com/justincastilla/deepagent_workshop_py

deepagents (Python)

github.com/langchain-ai/deepagents

Elastic Agent Builder

elastic.co/docs/.../elastic-agent-builder

ES|QL reference

elastic.co/docs/reference/.../esql

semantic_text + ELSER

elastic.co/docs/.../semantic-text

Questions?

